

MBLP136 – VERİ TABANI

2. Hafta: Veri Tabanı Mimarisi ve Veri Modellerine Giriş

Dr. Öğr. Üyesi Hasan Oğuz

İstanbul Okan Üniversitesi
Meslek Yüksekokulu

Veri Tabanı Kavramı

1. Veri Tabanı (Database) Nedir?

En temel ifadeyle veri tabanı, birbiriyle ilişkili verilerin mantıksal bir bütünlük içinde, sistematik olarak depolandığı dijital yapıdır.

- **Veri vs. Bilgi:** Ham gerçeklerin (data), anlamlı bir çıktıya (information) dönüşmesi sürecindeki en kritik duraktır.
- **Yapılandırılmış Veri:** Kaotik dosya yığınlarının aksine, veri tabanları belirli bir *şema* (schema) dahilinde çalışır.

Teknik Tanım

Veri tabanı, verilerin sadece depolandığı bir yer değil; veriler arasındaki bağıntıların ($R \subseteq A \times B$) ve kısıtlamaların (constraints) tanımlandığı bir sistemdir.

2. İlişkisel Model (Relational Model)

Günümüzde en yaygın kullanılan model olan "İlişkisel Model", veriyi tablolar (relations) halinde organize eder.

- **Tablo (Table/Relation):** Satırlar (tuples) ve sütunlardan (attributes) oluşur.
- **Birincil Anahtar (Primary Key):** Her bir satırı benzersiz (unique) kılan kimlik bilgisidir.
- **İlişki (Relationship):** Tabloların birbirine yabancı anahtarlar (Foreign Keys) üzerinden bağlanmasıdır.

Örnek: Bir "Öğrenci" tablosu ile "Dersler" tablosu, "Notlar" tablosu üzerinden ilişkilendirilir.

İlişkisel Model Örneği

İlişkisel Model Uygulaması: Tablo Entegrasyonu

1. Öğrenci Tablosu (Varlıklar)

OgrID (PK)	Ad	Soyad
101	Hasan	Oğuz
102	Ayşe	Demir
103	Can	Yılmaz

2. Dersler Tablosu (Varlıklar)

DersKod (PK)	Ders Adı
MBLP136	Veri Tabanı
MDDE102	Sinyal İşleme

3. Notlar Tablosu (İlişki / Bridge)

NotID	OgrID (FK)	DersKod (FK)	Puan
1	101	MBLP136	95
2	101	MDDE102	88
3	102	MBLP136	70

İlişkisel Mantık

Notlar tablosu, doğrudan isim saklamak yerine **OgrID** ve **DersKod** yabancı anahtarlarını (FK) kullanarak diğer tablolara referans verir.

Veri Bütünlüğü

105 ID'li bir öğrenci sisteme kayıtlı değilse, *Notlar* tablosuna bu ID ile giriş yapılamaz.

İlişkisel Model: Kütüphane Otomasyonu

Örnek 2: Veri Tekrarını Önlemek (Normalization)

Kitaplar Tablosu

KitapID (PK)	Kitap Adı	Yazar
B01	Veri Tabanı	İhsan Özer
B02	Sinyaller	S. Haykin
B03	Newton Mekaniği	A.P. French

Üyeler Tablosu

UyeID (PK)	Ad	Soyad
M500	Hasan	Oğuz
M501	Elif	Ak

Ödünç Alma (İşlem) Tablosu

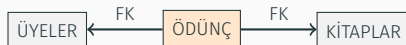
İşlemID	KitapID (FK)	UyeID (FK)	Tarih
1	B01	M500	14.02.2026
2	B01	M501	15.02.2026
3	B03	M500	16.02.2026

Mantıksal Çıkarım

"Ödünç" tablosunda kitabın adını veya üyenin soyadını tekrar yazmıyoruz. Sadece **B01** ve **M500** anahtarlarını tutarak:

- Depolama alanından tasarruf ediyoruz.
- Veri güncellendiğinde (örneğin üye soyadı değiştiğinde) sadece tek bir yerden değişiklik yapıyoruz.

Bu yapı, verinin **Tutarlılığını (Consistency)** nasıl korur?



- **Silme Kısıtı (On Delete):** Bir üyenin üzerinde iade edilmemiş kitap varken üyelik kaydının silinmesini engelleriz.
- **Güncelleme Kısıtı (On Update):** Kitap ID'si değişirse, tüm ödünç kayıtlarında otomatik güncellenmesini (Cascade) sağlayabiliriz.

SQL Kısıtlamaları (Constraints)

Veri Bütünlüğü ve Kısıtlamalar (Constraints)

Veri tabanındaki verilerin doğruluğunu, tutarlılığını ve güvenilirliğini sağlamak için kullanılan kurallardır.

- **NOT NULL:** Sütunun boş bırakılamayacağını garanti eder.
- **UNIQUE:** Sütundaki tüm değerlerin benzersiz olmasını sağlar.
- **PRIMARY KEY (PK):** *NOT NULL* ve *UNIQUE* kısıtlamalarının birleşimidir; satırı tekilleştirir.
- **FOREIGN KEY (FK):** İki tablo arasındaki ilişkiyi kurar ve referansal bütünlüğü sağlar.
- **CHECK:** Sütundaki değerlerin belirli bir koşulu sağlamasını şart koşar (Örn: *Puan* $> = 0$).

Referansal Bütünlük Kuralı

Bir tabloda yabancı anahtar (FK) olarak tanımlanan bir değer, referans verdiği tablonun birincil anahtarında (PK) mutlaka mevcut olmalıdır.

SQL Uygulaması: İlişkili Tablo Oluşturma

Kütüphane örneğimizdeki **Odunc** tablosunu, kısıtlamalarla birlikte tanımlayalım:

T-SQL / MySQL Ortak Yazımı

```
CREATE TABLE Odunc (  
    IslemID INT PRIMARY KEY,  
    KitapID CHAR(3) NOT NULL,  
    UyeID CHAR(4) NOT NULL,  
    Tarih DATE DEFAULT GETDATE(),  
    -- İlişki Tanımlamaları:  
    FOREIGN KEY (KitapID) REFERENCES Kitaplar(KitapID),  
    FOREIGN KEY (UyeID) REFERENCES Uyeler(UyeID)  
    ON DELETE CASCADE -- Üye silinirse ödünç kaydı da silinsin  
    ON UPDATE CASCADE -- Üye ID değişirse burada da güncellensin  
);
```

Cascade Mekanizması

ON DELETE / UPDATE CASCADE Mantığı

Referans alınan (Ebeveyn) tablodaki bir kayıt silindiğinde veya güncellendiğinde, bu durumun ilişkili (Çocuk) tabloya nasıl yansıtılacağını belirleriz:

1. **CASCADE:** Ebeveyn tabloda yapılan işlem (silme/güncelleme) otomatik olarak çocuk tabloya da yansıtılır.
2. **SET NULL:** Ebeveyn kayıt silindiğinde, çocuk tablodaki ilgili alan **NULL** yapılır (Eğer izin veriliyorsa).
3. **NO ACTION / RESTRICT:** Eğer ilişkili kayıt varsa, ebeveyn tablodaki kaydın silinmesine izin verilmez (En güvenli yöntemdir).

Kritik Senaryo

Eğer bir üyenin kütüphaneye borcu (iade etmediği kitap) varsa, **RESTRICT** kullanarak üyelik kaydının yanlışlıkla silinmesini engelleyebiliriz.

3. Veri Tabanı Yönetim Sistemi (VTYS/DBMS)

Veri tabanı ile kullanıcı/uygulama arasında yer alan yazılım katmanıdır. Temel görevleri şunlardır:

1. **Veri Tanımlama (DDL):** Veri yapısının oluşturulması ve değiştirilmesi.
2. **Veri Manipülasyonu (DML):** Veri ekleme, silme ve güncelleme işlemleri.
3. **Eşzamanlılık Kontrolü:** Aynı anda binlerce kullanıcının tutarlı bir şekilde veriye erişmesi.
4. **Güvenlik ve Yedekleme:** Verinin yetkisiz erişime karşı korunması ve felaket kurtarma senaryoları.

Önemli

Excel bir veri tabanı değildir; çünkü eşzamanlı erişim, veri bütünlüğü (integrity) ve karmaşık ilişki yönetimi yetenekleri sınırlıdır.

VTYS Mimarisi

Üç Seviyeli ANSI-SPARC Mimarisi

Veri tabanı sistemlerinde veriyi kullanıcıdan soyutlamak için üç temel seviye kullanılır:

1. **Dış Seviye (External Level):** Kullanıcıların veya uygulamaların gördüğü kısım (Görünümler/Views).
2. **Kavramsal Seviye (Conceptual Level):** Tüm veri tabanının mantıksal yapısı. "Hangi veriler tutulacak?" ve "Aralarındaki ilişkiler neler?" sorularına yanıt verir.
3. **İç/Fiziksel Seviye (Internal Level):** Verinin disk üzerinde nasıl saklandığı, indeksleme ve depolama detayları.

Veri Bağımsızlığı

Bu mimari sayesinde fiziksel yapı değişse bile (örn. HDD'den SSD'ye geçiş), mantıksal yapı ve uygulamalar etkilenmez.

Veri Modelleri

Bir veri tabanı tasarlanırken şu aşamalardan geçilir:

- **Kavramsal Model:** İş mantığını temsil eder (ER Diyagramları). Yazılımdan bağımsızdır.
- **Mantıksal Model:** Verinin hangi yapıda (ilişkisel, nesne yönelimli vb.) tutulacağını belirler.
- **Fiziksel Model:** Veri tabanı motoruna (SQL Server, MySQL) özgü teknik tanımlamalar.

Varlık-İlişki (ER) Modeli: Temel Bileşenler

Varlık (Entity) ve Öznitelik (Attribute)

- **Varlık (Entity):** Hakkında veri sakladığımız nesne veya kavramlardır (Örn: Öğrenci, Ders, Personel).
- **Öznitelik (Attribute):** Varlığı tanımlayan özelliklerdir (Örn: Öğrenci No, Adı, Soyadı).

Öznitelik Türleri:

- *Anahtar Öznitelik (Key):* Varlığı benzersiz kılan (Örn: TC No).
- *Çok Değerli (Multivalued):* Birden fazla değer alabilen (Örn: Telefon numaraları).
- *Türetilen (Derived):* Diğer verilerden hesaplanan (Örn: Doğum tarihinden hesaplanan Yaş).

Varlık ve Öznitelik Örnekleri

Örnek 1: Personel Varlığı

Bir işletmedeki çalışanları temsil eden varlık yapısı:

- **Anahtar (Key):** PersonelNo veya TC No.
- **Basit:** Ad, Soyad, Maaş.
- **Çok Değerli:** *Uzmanlık Alanları* (Örn: SQL, Java, Network).
- **Türetilen:** *Kıdem Yılı* (İşe Giriş Tarihi üzerinden hesaplanır).

Örnek 2: Araç Varlığı

Bir filo yönetim sistemindeki araç varlığı yapısı:

- **Anahtar (Key):** Plaka veya Şasi No.
- **Basit:** Marka, Model, Yakıt Türü.
- **Çok Değerli:** *Aksesuar Listesi* (Sunroof, GPS, Park Sensörü).
- **Türetilen:** *Araç Yaşı* (Model Yılı üzerinden hesaplanır).

Tasarımsal İpucu: Çok değerli öznitelikler, ilişkisel modele geçilirken genellikle ayrı bir tabloya taşınır.

Varlık ve Özniteliklerin Sınıflandırılması

Örnek 1: Personel Varlığı Analizi

Öznitelik Türü	Öznitelik Adı	Açıklama / Örnek
Anahtar (Key)	<u>PersonelNo</u>	Benzersiz kimlik (ID: 1001)
Basit (Simple)	Ad, Soyad	Tek bir veri içerir (Hasan, Oğuz)
Çok Değerli (Multivalued)	Uzmanlık	Birden fazla değer (SQL, Python, LaTeX)
Türetilen (Derived)	Kıdem Yılı	Fonksiyonel (İşe giriş tarihinden hesaplanır)

Örnek 2: Araç Varlığı Analizi

Öznitelik Türü	Öznitelik Adı	Açıklama / Örnek
Anahtar (Key)	<u>Plaka</u>	Benzersiz kimlik (34 ABC 123)
Basit (Simple)	Marka, Model	Tekil özellik (Volvo, XC90)
Çok Değerli (Multivalued)	Aksesuar	Liste şeklinde (Sunroof, Deri Koltuk)
Türetilen (Derived)	Araç Yaşı	Fonksiyonel (Model yılından hesaplanır)

- **Türetilen (Derived) Öznitelikler:** Veri tabanında fiziksel yer kaplamazlar. Sorgu anında hesaplanırlar (*Calculated Fields*).
- **Çok Değerli (Multivalued) Öznitelikler:** İlişkisel modelde "atomiklik" ilkesi gereği, bu tür veriler genellikle ayrı bir tabloda tutulur.
- **Bileşik (Composite) Öznitelikler:** Örn: "Adres" özneliği; Sokak, Mahalle, Şehir olarak parçalanabilir.

Akademik İpucu

İyi bir veri tabanı tasarımı, verinin en küçük (atomik) parçasına kadar bölünmesini gerektirir.

ER Modelinden İlişkisel Modele Dönüşüm

Bir ER diyagramını tablo yapısına dönüştürürken şu kuralları izleriz:

1. Her **Varlık (Entity)** için bir tablo oluşturulur.
2. Her **Anahtar Öznitelik**, tablonun **Primary Key (PK)** alanı olur.
3. **1:N ilişkilerde**, N tarafına **Foreign Key (FK)** eklenir.
4. **M:N ilişkilerde**, iki PK'yi içeren bir **Ara Tablo** oluşturulur.
5. **Çok değerli öznitelikler** ayrı tabloya taşınır.

Mantıksal Amaç

Tekrarı azaltmak ve veri bütünlüğünü garanti altına almak.

İlişkiler (Relationships) ve Dereceleri

Varlıklar arasındaki bağları tanımlarız:

- **Bire-Bir (1:1):** Bir bölümün sadece bir başkanı olabilir.
- **Bire-Çok (1:N):** Bir danışman hocanın birçok öğrencisi olabilir.
- **Çoğa-Çok (M:N):** Bir öğrenci birçok ders alabilir, bir dersin birçok öğrencisi olabilir.

Kritik Uyarı

İlişkisel veri tabanlarında (SQL Server vb.) Çoğa-Çok (M:N) ilişkiler doğrudan kurulamaz; "Ara Tablo" kullanılması zorunludur.

İlişki Dereceleri: Uygulama Örnekleri

İlişki Türleri ve Gerçek Hayat Senaryoları

İlişki Türü	Varlık A	Varlık B
Bire-Bir (1:1)	Ülke	Başkent
Bire-Bir (1:1)	Personel	Şirket Aracı (Özel tahsis)
Bire-Çok (1:N)	Müşteri	Siparişler
Bire-Çok (1:N)	Şehir	İlçeler
Çoğa-Çok (M:N)	Yazar	Kitaplar
Çoğa-Çok (M:N)	Hasta	Doktorlar

Mantıksal Sınav

"Bir öğrencinin sadece bir danışmanı olabilir, ancak bir danışmanın birçok öğrencisi vardır."

Bu ifade, Danışman → Öğrenci yönünde **1:N** bir ilişkidir.

M:N İlişkisi ve Ara Tablo Çözümü

M:N İlişkisinin Çözümlemesi (Ara Tablo Mantığı)

İlişkisel veri tabanlarında M:N bir bağıntıyı doğrudan kodlayamayız. Bu yapıyı iki adet **1:N** ilişkiye bölmek için bir "Ara Tablo" (Junction/Bridge Table) oluştururuz.

Senaryo: Öğrenci ve Dersler



- **Ders_Kayıt** tablosu, her iki tablonun **Primary Key (PK)** değerlerini **Foreign Key (FK)** olarak bünyesinde barındırır.
- Ek veriler (Örn: Vize notu, Final notu, Devamsızlık) bu ara tabloda saklanır.

Uygulamalı Örnek: Hastane Randevu Sistemi

Bir hastane veri tabanında Doktorlar ve Hastalar arasındaki M:N ilişkiyi inceleyelim:

Varlıklar:

- **Doktor:** Dr. Ahmet, Dr. Elif.
- **Hasta:** Hasan, Ayşe, Can.

İlişki:

- Bir doktor gün içinde birçok hastaya bakabilir.
- Bir hasta farklı uzmanlık alanlarındaki birçok doktora gidebilir.

Çözüm: Randevu Tablosu

- RandevuID (PK)
- DoktorID (FK)
- HastaID (FK)
- RandevuTarihi
- Şikayet Notu

Bu sayede M:N ilişki, iki adet 1:N ilişkiye indirgenmiş olur.

Fonksiyonel Bağımlılık (Functional Dependency)

Bir tabloda A alanı, B alanını belirliyorsa:

$$A \rightarrow B$$

Örnek:

$$OgrID \rightarrow (Ad, Soyad)$$

Bu şu anlama gelir:

- OgrID biliniyorsa, Ad ve Soyad tekil olarak belirlenir.
- Aynı OgrID için iki farklı isim olamaz.

Neden Önemli?

Normalizasyon kuralları fonksiyonel bağımlılıklar üzerinden tanımlanır.

Normalizasyon Seviyelerine Giriş

1NF (Birinci Normal Form)

- Tablodaki her hücre atomik olmalıdır.
- Çok değerli alan bulunmaz.

2NF

- 1NF sağlanmış olmalı.
- Kısmi bağımlılık olmamalı.

3NF

- Geçişli bağımlılık olmamalı.

Stratejik Amaç

Veri tekrarını ve güncelleme anomalilerini ortadan kaldırmak.

Kurumsal vs Web Startup Veri Mimarisi

Kurumsal Sistemler

- MS SQL Server
- T-SQL, Stored Procedure
- Rol bazlı yetkilendirme
- Transaction yönetimi

Bu Dersin Stratejisi

Her iki yaklaşımı da öğreneceğiz.

Web Startup Ekosistemi

- MySQL / MariaDB
- REST API entegrasyonu
- Hızlı prototipleme
- ORM (Laravel, Django)

Bir transaction ya tamamen gerçekleşir ya hiç gerçekleşmez.

ACID İlkeleri:

- **Atomicity** – Ya hep ya hiç.
- **Consistency** – Kurallar ihlal edilmez.
- **Isolation** – Eşzamanlı işlemler birbirini bozmaz.
- **Durability** – İşlem kalıcıdır.

Bankacılık Örneği

1000 TL transfer işleminde para iki hesaptan biri güncellenip diğeri güncellenmezse sistem tutarsız olur.

Haftalık Özet

2. Hafta Özeti: Mimariden Bütünlüğe

1. Veri Mimarisi (ANSI-SPARC)

- **Dış/Kavramsal/İç** seviyeler arası soyutlama.
- **Veri Bağımsızlığı:** Fiziksel değişimlerin mantıksal yapıyı bozmaması.

2. Modelleme Süreci

- **ER Modeli:** Gerçek dünyanın diyagramlarla tasviri.
- **İlişkisel Model:** Verinin tablolar, anahtarlar ve kısıtlamalarla ifadesi.

Öğrenim Çıktısı

Veri tabanı sadece veri saklamak için değil; veriyi **yapılandırmak, ilişkilendirmek** ve **korumak** içindir.

3. Veri Bütünlüğü (Integrity)

- **PK (Primary Key):** Satırı tekillleştirir.
- **FK (Foreign Key):** Tablolar arası köprü kurar.
- **Constraints:** İş kurallarını (***NOT NULL, UNIQUE, CHECK***) sisteme gömer.

4. Cascade Mekanizması

- **CASCADE** ile zincirleme işlem.
- **RESTRICT** ile veri güvenliği ve silme koruması.

2. Hafta Özeti: Veri Tabanı Tasarım Adımları ve ACID

Temel Tasarım Adımları:

- İhtiyaç analizi yap (Hangi veriler tutulacak?).
- ER modeli oluştur (Varlık, Öznitelik, İlişki).
- Anahtarları belirle (Primary Key).
- İlişkileri tanımla (Foreign Key).
- Fonksiyonel bağımlılıkları analiz et.
- Normalizasyon uygula (1NF $\rightarrow$ 3NF).
- Fiziksel modele ve SQL tablolarına dönüştür.

Transaction ve ACID:

- **Atomicity:** İşlem ya tamamen gerçekleşir ya hiç gerçekleşmez.
- **Consistency:** Veri kuralları her zaman korunur.
- **Isolation:** Eşzamanlı işlemler birbirini bozmaz.
- **Durability:** Tamamlanan işlem kalıcıdır.

Gelecek Haftaya Hazırlık

Gelecek Hafta: SQL'e Giriş ve Uygulama

- **Modelden Tabloya:** Tasarladığımız ER diyagramlarını matematiksel bir disiplinle **İlişkisel Tablolara** dönüştüreceğiz.
- **D207 Laboratuvar Uygulaması:** İlk SQL komutlarımızı (*CREATE, DROP, ALTER*) canlı sistemde koşturacağız.
- **Kurumsal Standart:** MS SQL Server ve **SSMS** (Management Studio) arayüzünde uzmanlaşacağız.
- **Hızlı Prototipleme:** Web tabanlı uygulamalarda çeviklik kazanmak adına **XAMPP**, **MySQL** ve **MySQL Workbench** ile paralel senaryolar uygulayacağız.

Sektörel Odak

Kurumsal veri yönetimi ve T-SQL mimarisi.

Web & Startup Odağı

Hızlı uygulama geliştirme ve Cross-platform çözümler.

Not: Kendi cihazınızla geliyorsanız lütfen SQL Server Express ve XAMPP kurulumlarını tamamlayınız.